

**Coding Challenge**[← Back](#)

## QUESTIONS

**Q1** ✓  
String Character Frequency  
Counter  
10 marks**Q2** ✓  
Vowel and Consonant Classifier  
with Statistics  
10 marks**Q3** ✓  
String Compression and Run-  
Length Encoding  
10 marks**Q4** ✓  
Advanced Text Statistics  
Analyser  
10 marks

4/4 answered

**Q1 String Character Frequency Counter**

10 marks

Write a Python program that accepts a string from the user and counts the frequency of each unique character (case-insensitive). Display the results in the format: character → count. Sort the output alphabetically by character.

Constraints:

- Ignore spaces in the count.
- Treat uppercase and lowercase as the same character.
- Sort output alphabetically.

Skills Assessed:

- String iteration, lower(), conditionals
- Dictionary or manual counting
- Sorting using loops or sorted()

Input: Hello World

output:

d → 1  
e → 1  
h → 1  
l → 3  
o → 2  
r → 1  
w → 1

**Your Code**

```
1 s = input().lower()
2
3 freq = {}
4
5 for ch in s:
6     if ch != ' ':
7         if ch in freq:
8             freq[ch] += 1
9         else:
10            freq[ch] = 1
11
12 for ch in sorted(freq):
13     print(ch, freq[ch])
```

**Input**

Hello World

**Output**

```
d 1
e 1
h 1
l 3
o 2
r 1
w 1
```

 **Submitted**

Counter  
10 marks**Q2** ✓  
Vowel and Consonant Classifier  
with Statistics  
10 marks**Q3** ✓  
String Compression and Run-  
Length Encoding  
10 marks**Q4** ✓  
Advanced Text Statistics  
Analyser  
10 marks

4/4 answered

frequent vowel, (c) the least frequent consonant, and (d) the ratio of vowels to consonants as a decimal rounded to 2 places.

Constraints:

- Case-insensitive analysis.
- Ignore spaces and digits.
- Handle sentences with no vowels or no consonants gracefully.

Skills Assessed:

- Character classification with conditionals
- Counting and comparison logic
- Arithmetic and rounding

Sample Input :

Python Programming

Expected Output : Vowels: 5 | Consonants: 12 Most frequent vowel: o (2) Least frequent consonant: y (1) Vowel-to-Consonant Ratio: 0.42

**Your Code**

```
1 s = input().lower()
2
3 vowels = 'aeiou'
4 vowel_count = 0
5 consonant_count = 0
6
7 vf = {}
8 cf = {}
9
10 for ch in s:
11     if ch.isalpha():
12         if ch in vowels:
13             vowel_count += 1
14             if ch in vf:
15                 vf[ch] += 1
16         else:
17             vf[ch] = 1
18     else:
19         consonant_count += 1
20         if ch in cf:
21             cf[ch] += 1
22         else:
23             cf[ch] = 1
24
25 print('Vowels:', vowel_count)
26 print('Consonants:', consonant_count)
27
28 if vf:
29     mv = max(vf, key=vf.get)
30     print('Most frequent vowel:', mv, '(', vf[mv], ')', sep='')
31 else:
32     print('Most frequent vowel: None')
33
34 if cf:
35     lc = min(cf, key=cf.get)
36     print('Least frequent consonant:', lc, '(', cf[lc], ')', sep='')
37 else:
38     print('Least frequent consonant: None')
39
40 if consonant_count > 0:
41     print('Vowel-to-Consonant Ratio:', round(vowel_count / consonant_count, 2))
42 else:
43     print('Vowel-to-Consonant Ratio: Undefined')
```

**Input**

Python Programming

**Output**

```
Vowels: 4
Consonants: 13
Most frequent vowel:o(2)
Least frequent consonant:y(1)
Vowel-to-Consonant Ratio: 0.31
```

✓ Submitted



Counter  
10 marksQ2  
Vowel and Consonant Classifier  
with Statistics  
10 marksQ3  
String Compression and Run-  
Length Encoding  
10 marksQ4  
Advanced Text Statistics  
Analyser  
10 marks

4/4 answered

replacing consecutive repeated characters with character+count. (b) Decompresses the encoded string back to the original. (c) If the compressed string is not shorter, returns the original.

Constraints:

- Single characters without count still output as single char.
- Handle empty strings.
- Decompressor should handle all valid RLE strings.

Skills Assessed:

- String traversal and comparison
- Count accumulation logic
- String building and integer conversion

Sample Input :

aabbbccddddee

Expected Output

Compressed: a2b3c2d4e2

Decompressed: aabbbccddddee

For input 'abc': Compression not beneficial.

Original:

## Your Code

```
1 s = input()
2
3 if s == '':
4     print("Compressed:")
5     print("Decompressed:")
6 else:
7     compressed = ''
8     count = 1
9
10    for i in range(1, len(s)):
11        if s[i] == s[i - 1]:
12            count += 1
13        else:
14            compressed += s[i - 1]
15            if count > 1:
16                compressed += str(count)
17            count = 1
18
19    compressed += s[-1]
20    if count > 1:
21        compressed += str(count)
22
23    if len(compressed) < len(s):
24        print("Compressed:", compressed)
25
26    decompressed = ''
27    i = 0
28    while i < len(compressed):
29        ch = compressed[i]
30        i += 1
31
32        num = ''
33        while i < len(compressed) and compressed[i].isdigit():
34            num += compressed[i]
35            i += 1
36
37        if num == '':
38            decompressed += ch
39        else:
40            decompressed += ch * int(num)
41
42    print("Decompressed:", decompressed)
43 else:
44     print("Compression not beneficial.")
45     print("Original:", s)
```

## Input

aabbbccddddee

## Output

```
Compressed: a2b3c2d4e2
Decompressed: aabbbccddddee
```



```
2
3 text = input()
4
5 stop = ['the', 'is', 'a', 'an', 'in', 'of', 'and', 'to', 'for']
6
7 paragraphs = len([p for p in text.split('\n') if p.strip() != ''])
8 sentences = len(re.findall(r'[.!?]', text))
9 if sentences == 0:
10     sentences = 1
11
12 words = re.findall(r'\b[a-zA-Z]+\b', text)
13 total_words = len(words)
14
15 total_length = 0
16 freq = {}
17 syllables = 0
18
19 for w in words:
20     word = w.lower()
21     total_length += len(word)
22
23     if word not in stop:
24         if word in freq:
25             freq[word] += 1
26         else:
27             freq[word] = 1
28
29     count = 0
30     prev = False
31     for ch in word:
32         if ch in 'aeiou':
33             if not prev:
34                 count += 1
35             prev = True
36         else:
37             prev = False
38     if count == 0:
39         count = 1
40     syllables += count
41
42 avg_word = round(total_length / total_words, 2) if total_words else 0
43 avg_sentence = round(total_words / sentences, 2) if sentences else 0
44
45 score = round(206.835 - 1.015 * (total_words / sentences) - 84.6 * (syllables / total_words), 2)
46
47 print('Words:', total_words)
48 print('Sentences:', sentences)
49 print('Paragraphs:', paragraphs)
50 print('Avg Word Length:', avg_word)
51 print('Avg Sentence Length:', avg_sentence)
52
53 top = sorted(freq.items(), key=lambda x: (-x[1], x[0]))[:5]
54 print('Top 5 Words:')
55 for w, c in top:
56     print(w, c)
57
58 print('Flesch Reading Ease:', score)
```

**Input**

Python is a powerful programming language

**Output**

```
Words: 6
Sentences: 1
Paragraphs: 1
Avg Word Length: 6.0
Avg Sentence Length: 6.0
Top 5 Words:
```

Submitted